

MapReduce Computation Model

Baptiste Jeudy

`baptiste.jeudy@univ-st-etienne.fr`

Laboratoire Hubert Curien,
Université Jean-Monnet Saint-Etienne.

Master MLDM DSC

Outline

- 1 Introduction: Big Data
- 2 MapReduce Framework
 - Overview
 - Hardware architecture
 - Distributed File System
- 3 MapReduce Programming Model
 - Map
 - ReduceByKey
 - Generalization
- 4 Example: Word Count
 - FlatMap

Big Data

Big Data definition

Data too large to fit in memory or on a single computer

Problems with big data:

- Disk are slow
 - Google example: 50 billion web pages $\times$ 20KB = 1000TB
 - HDD bandwidth $\approx$ 100MB/s (SSD: 300MB/s)
 - $\Rightarrow$ 4 months just to read the data from one disk

Big Data

Big Data definition

Data too large to fit in memory or on a single computer

Problems with big data:

- Disk are slow
 - Google example: 50 billion web pages $\times$ 20KB = 1000TB
 - HDD bandwidth $\approx$ 100MB/s (SSD: 300MB/s)
 - $\Rightarrow$ 4 months just to read the data from one disk
- $\Rightarrow$ use several nodes (computers)

Problems with Big Data

⇒ use several nodes (computers), but then:

- Communication between nodes is slow:
 - bandwidth/latency of network several order of magnitude lower than of memory (RAM or processor cache)
 - e.g., bandwidth: 100 Gb/s for RAM vs. 1-10 Gb/s for network
 - latency: 10^{-7} s for RAM vs. 10^{-3} s for network
- Failure of nodes / network. Ex:
 - A node may fail after 3 years $\approx$ 1000 days
 - Google had $\approx$ 2M servers in 2013
 - $\Rightarrow$ 2000 failures / day !
- Parallelism is difficult

Parallelism is difficult

Shared variable

- initial value: $c:=0$
- node 1: compute $c:=c+3$
- node 2: compute $c:=c+5$

Final expected value of c : 8.

Parallelism is difficult

Shared variable

- initial value: $c:=0$
- node 1: compute $c:=c+3$
- node 2: compute $c:=c+5$

Final expected value of c : 8.

On each node:

- Read c in shared memory
- Compute $c+..$
- Write back the result in shared memory c
- What could possibly go wrong?

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute $c+3$		0

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute $c+3$		0
3	write c		3

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute $c+3$		0
3	write c		3
4		read c (=3)	3

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
------	--------	--------	-------------------

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0
3		read c (=0)	0

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0
3		read c (=0)	0
4	write c		3

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0
3		read c (=0)	0
4	write c		3
5		compute c+5	3

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0
3		read c (=0)	0
4	write c		3
5		compute c+5	3
6		write c	5

Parallelism is difficult

Expected execution

Time	Node 1	Node 2	Shared variable c
1	read c		0
2	compute c+3		0
3	write c		3
4		read c (=3)	3
5		compute c+5	3
6		write c	8

Possible execution

Time	Node 1	Node 2	Shared variable c
1	read c (=0)		0
2	compute c+3		0
3		read c (=0)	0
4	write c		3
5		compute c+5	3
6		write c	5

Final value of c: 5!

Called a race condition

Parallelism is difficult

- Bugs (race conditions)
- Needs synchronization (mutual exclusion)
- Could lead to deadlocks, other bugs (e.g., if a node dies)

Difficult to find bugs, to reproduce bugs and to correct them.

1 Introduction: Big Data

2 MapReduce Framework

- Overview
- Hardware architecture
- Distributed File System

3 MapReduce Programming Model

4 Example: Word Count

Google MapReduce

- **MapReduce framework (2004):** adress these problems.
 - Slow communications : $\Rightarrow$ **minimize them!** (local computations)
 - Failure of nodes/network : $\Rightarrow$ **transparent redundancy**
 - Parallelism is difficult : $\Rightarrow$ **simple framework**
- Proposed in: Jeffrey Dean, Sanjay Ghemawat
MapReduce: Simplified Data Processing on Large Clusters,
Symposium on Operating Systems Design and Implementation.
www.usenix.org/legacy/publications/library/proceedings/osdi04/tech/full_papers/dean/dean_html/index.html
- Based on:
 - Distributed file system: GFS (Google File System).
 - MapReduce programming model: elegant solution to distributed computing

Apache Hadoop / Spark

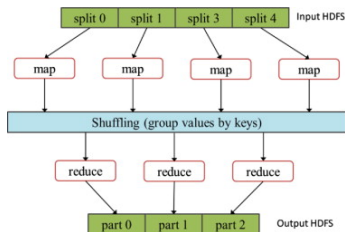


- Apache Hadoop (2011): free software implementation of MapReduce
- Uses Hadoop Distributed File System (HDFS)
- <http://hadoop.apache.org/>
- Apache Spark: generalization of MapReduce.
- <http://spark.apache.org/>

MapReduce Overview

Distributed storage and distributed processing on thousands of nodes

- Data is distributed on nodes
- Data is duplicated on several nodes (failure tolerance)
- No shared memory (only files)
- Large data files (GB to TB)
- Bring the program to the data
- Write-once-read-many access for files: only append and truncate operation allowed on files (simplify coherency issues)



MapReduce Overview

- Not suitable for every kind of computation
 - bottleneck of data transmission on network
 - starting time (few sec) for every task
 - $\Rightarrow$ other approach (like GPU computing) may be more suitable

MapReduce Overview

- 3 components in the framework:
 - **Hardware**
 - nodes
 - disks
 - network
 - **Software**: processes running on each node that manage:
 - distributed and redundant file system;
 - computations;
 - communication between nodes.
 - **Programming model** (how to write programs for MapReduce)
- Important to understand each component and their relations

1 Introduction: Big Data

2 MapReduce Framework

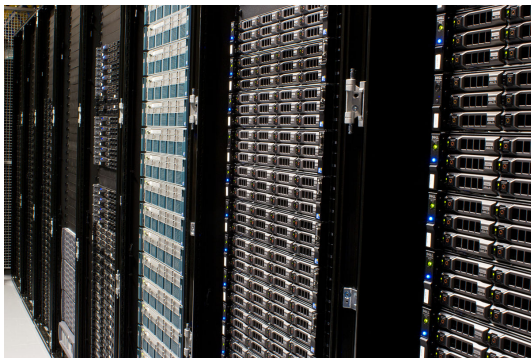
- Overview
- Hardware architecture
- Distributed File System

3 MapReduce Programming Model

4 Example: Word Count

Nodes

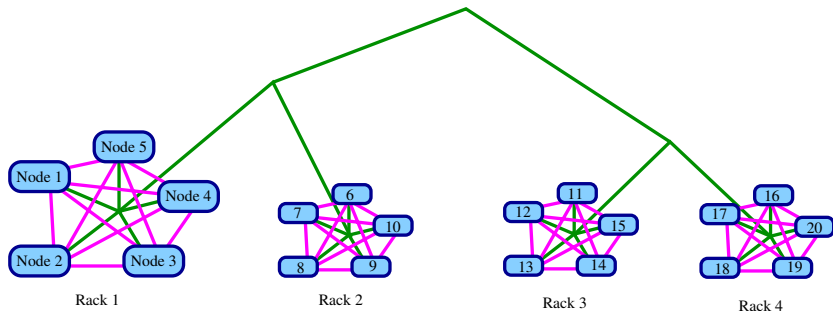
- Commodity Linux nodes: CPU, local memory and storage
- Interconnected by a commodity network
- Each node used for both storage and computation



By Victorgrigas (Own work) [CC BY-SA 3.0] via Wikimedia Commons

Cluster Architecture

- Tree like
- bandwidth between nodes depends on their location
 - e.g., 1Gb/s in the same rack between any two nodes
 - Shared 10Gb/s otherwise



- 1 Introduction: Big Data
- 2 MapReduce Framework
 - Overview
 - Hardware architecture
 - Distributed File System
- 3 MapReduce Programming Model
- 4 Example: Word Count

Distributed File System (without replication)

Simple distributed file system

- 3 nodes
- 4 files A, B, C and D
- No replication

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
 - node failure $\Rightarrow$ data lost

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
 - node failure $\Rightarrow$ data lost
 - Distributed computing not efficient:

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
 - node failure $\Rightarrow$ data lost
 - Distributed computing not efficient:
 - e.g., computation with data from file B

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
 - node failure $\Rightarrow$ data lost
 - Distributed computing not efficient:
 - e.g., computation with data from file B
 - $\Rightarrow$ need to transfer data from node 2 to node 1 and 3

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
- node failure $\Rightarrow$ data lost
- Distributed computing not efficient:
- e.g., computation with data from file B
 - $\Rightarrow$ need to transfer data from node 2 to node 1 and 3
 - But data read and transfer takes (lot of) time:
If file is big, data read/transfer takes more time than computation!

Node 1



Node 2



Node 3



Distributed File System (without replication)

Simple distributed file system

- 3 nodes
 - 4 files A, B, C and D
 - No replication
- Not failure proof:
- node failure $\Rightarrow$ data lost
- Distributed computing not efficient:
- e.g., computation with data from file B
 - $\Rightarrow$ need to transfer data from node 2 to node 1 and 3
 - But data read and transfer takes (lot of) time:
If file is big, data read/transfer takes more time than computation!
- $\Rightarrow$ **not suitable for big data**

Node 1



Node 2

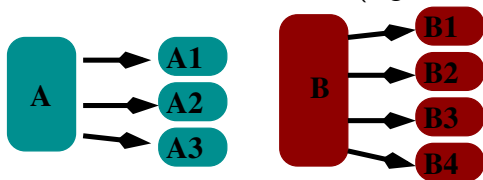


Node 3

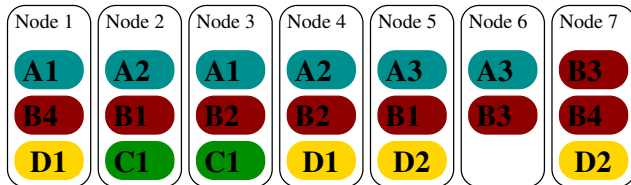


Distributed File System (GFS or HDFS)

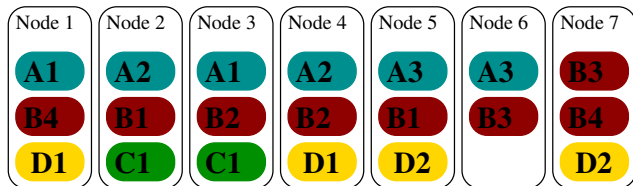
- Large files ($> \text{GB}$)
- Each file divided into blocks (e.g., 64MB)



- Each block replicated (e.g., $\times 3$, $\times 4$)
- Each replicated block on a different node / rack: redundancy



Distributed File System (GFS or HDFS)



- Failure of one node/rack is manageable (because of replication)
- Distributed computing is efficient:
 - e.g., computation with data from file B
 - Computation on B1 done on node 2 or 5
 - B2 on node 3 or 4
 - B3 on node 6 or 7
 - B4 on node 7 or 1
 - no data transfert!

Distributed File System (GFS or HDFS)

- One "Master Node" (Name Node in HDFS): manage the file system namespace (hierarchy) and mapping of blocks to Data Nodes (Master node may be duplicated)
- Many "Data Nodes" (one per node): store blocks of data (content of files)
- To read/write a file:
 - Connect to the Name Node to know where the file is stored
 - Direct connection to the correct Data Node to access file data
- Master Node checks regularly status of Data Nodes

Distributed File System (GFS or HDFS)

- Every block of data is duplicated on several Data Nodes (redundancy)
- Each data node is also a compute node
- $\Rightarrow$ Bring the computation close to the data

- 1 Introduction: Big Data
- 2 MapReduce Framework
- 3 MapReduce Programming Model**
 - Map
 - ReduceByKey
 - Generalization
- 4 Example: Word Count

MapReduce Framework

- All data (input and output) are **lists**
 - Elements of lists can be of any type: number, text, list, vector,...
- 2 main operations in MapReduce framework:
 - **map**: independent computations done in parallel on each element
 - **reduceByKey**: Computation that need several elements of the lists.
 - **shuffle**: group elements of the lists in groups
 - **reduce**: computation on each group in parallel
- **map** used for independent computations on each element of lists
 - ex: input: list of numbers; output: list of the square of each number
- **reduceByKey** if several elements of the list are needed for a computation.
 - ex: input: list of (student_name, evaluation) with several evaluations for each student. output: list of (student_name, average_mark).

- 1 Introduction: Big Data
- 2 MapReduce Framework
- 3 MapReduce Programming Model
 - Map
 - ReduceByKey
 - Generalization
- 4 Example: Word Count

MapReduce Framework : Map

- Input is a list L of objects or values
 - object can be of any type: number, text, list, vector,...
- User (programmer) must provide a function f
 - This function takes as input an object o of the list
 - and return one new object $f(o)$
- Result:
 $L = [o_1, o_2, o_3, \dots] \Rightarrow \text{map}(f) \Rightarrow \text{result} = [f(o_1), f(o_2), f(o_3), \dots]$
- notation (python style): $\text{result} = L.\text{map}(f)$
- Very efficient:
 - Suppose list L is already distributed on several nodes
 - $\Rightarrow$ computations of f done in parallel on all nodes
 - result is stored locally on each node: the result list is distributed!
 - no communication (which are slow)!
- Variation: flatmap (see latter)
- ex: list of (customer, product, price): apply local taxes.

- 1 Introduction: Big Data
- 2 MapReduce Framework
- 3 MapReduce Programming Model
 - Map
 - ReduceByKey
 - Generalization
- 4 Example: Word Count

MapReduce Framework : ReduceByKey

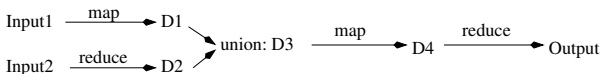
- Input is a list L of pairs (key, value).
 - key and value can be of any type: number, text, list, vector,...
- User (programmer) must provide a function f
 - This function takes as input two values $v1$ and $v2$ of the list
 - and return one new value $r = f(v1, v2)$
 - This function "reduce" several values into one.
- ReduceByKey: two sub-steps:
 - shuffle: group pairs with the same key
 - reduce: apply f on values with the same key (computation tree) until one value for each key.
 - result of the ReduceByKey: one pair (key,value) for each key.
- Notation : $\text{result} = L.\text{reduceByKey}(f)$
- ex: list of (customer, product, price): compute total price for each customer..

MapReduce Framework : ReduceByKey

- Distributed computations:
 - Suppose list **L** is already distributed on several nodes
 - To apply **f**: value associated to the same key must be send on the same node (**shuffle** sub-step)
 - $\Rightarrow$ communication between nodes : can be slow
- limitation:
 - function **f** must be **associative and commutative**
 - otherwise: result is not deterministic

Generalization

- MapReduce framework (2004): One map followed by one reduce
Input list $\Rightarrow$ map $\Rightarrow$ reduce $\Rightarrow$ Output list
- Generalization: several map/reduce in any order.
 - Input $\Rightarrow$ map $\Rightarrow$ reduce $\Rightarrow$ reduce $\Rightarrow$ Output
 - Input $\Rightarrow$ reduce $\Rightarrow$ map $\Rightarrow$ reduce $\Rightarrow$ Output
 - ...
- Graph representation for more complex workflows (even with loops)



- We will use this generalization

- 1 Introduction: Big Data
- 2 MapReduce Framework
- 3 MapReduce Programming Model
- 4 Example: Word Count
 - FlatMap

Example: Word Count

- Input: text file. List of lines.

```
L = ["When Mr.  Bilbo Baggins of Bag End announced that he would  
shortly",  
"be celebrating his eleventy-first birthday with a party of special",  
"magnificence, there was much talk and excitement in Hobbiton."  
...]
```

The Lord of the Rings, J. R. R. Tolkien

- Output: we want the frequency of each word in the file

Example: Word Count

- Input: text file. List of lines.

```
L = ["When Mr.  Bilbo Baggins of Bag End announced that he would  
shortly",  
"be celebrating his eleventy-first birthday with a party of special",  
"magnificence, there was much talk and excitement in Hobbiton."  
...]
```

The Lord of the Rings, J. R. R. Tolkien

- Output: we want the frequency of each word in the file

```
[...  
("announced", 1),  
...  
("party", 9),  
...  
("birthday", 11),  
...  
("Bilbo", 81),  
...  
("and", 390),  
("the", 408)]
```

MapReduce Framework : flatMap

Split the lines into words:

- Function `f` takes a line as input and output the list of words in the line.
- $L2 = L.map(f)$: list of list of words
 - Not what we want: (we want a list of words)
 - $\Rightarrow$ `flatMap` operation

flatMap (variation of map)

- Input is a list `L` of `objects` or values
- User (programmer) must provide a function `f`
- Result:

$L = [o1, o2, o3, \dots] \Rightarrow \text{flatMap}(f) \Rightarrow \text{result} = f(o1) + f(o2) + f(o3) + \dots$

where `+` is list concatenation
- notation (python style): `result = L.flatMap(f)`

Back to our word count example

Split the lines into words:

- Function `f` takes a line as input and output the list of words in the line. (with a count of 1).
- `L2 = L.flatMap(f)` : list of words in the text
- `f` is very simple, in Python:

```
def f(x):
    result = []
    for w in x.split():
        result.append((w,1))
    return result
```

flatMap Output: L2

<code>L.flatMap(f)</code>	⇒	L2
<code>f("Bilbo was ... peculiar,")</code>	⇒	<code>("Bilbo",1), ("was",1), ("very",1), ..., ("peculiar",1)</code>
<code>f("and had been ... years,")</code>	⇒	<code>("and",1), ("had",1), ("been",1), ..., ("years",1)</code>
<code>...</code>	⇒	<code>...</code>

Word Count Example: ReduceByKey

Output of flatMap: L2

```
L2=[ ("Bilbo",1), ("was",1), ("very",1), ..., ("peculiar",1), ("and",1),
      ("had",1), ("been",1), ..., ("years",1) ... ]
```

- We want to compute the sum for identical words:
⇒ use reduceByKey !
- $R = L2.reduceByKey(g)$ (need to find the correct g)

L2.reduceByKey(g)

L2 sorted by key after shuffle	Reduction with g	result R we want
("Bilbo",1),("Bilbo",1),("Bilbo",1)	$\Rightarrow^g$	("Bilbo",3)
("was",1), ("was",1)	$\Rightarrow^g$	("was",2)
("very",1),("very",1),("very",1)	$\Rightarrow^g$	("very",3)
...	...	...

- g is just a sum !

```
def g(v1,v2):
  return v1 + v2
```

Example 2: min/max

- Input: rows of a database

productId	price	weight
1	14	1.5
2	9	0.7
3	10	0.2

represented by a list of triplets $l = [(1, 14, 1.5), (2, 9, 0.7), \dots]$

- Wanted Output: the min and max values of price and weight.

Solution: $L1 = l.map(f)$ (or is it flatMap ?); $Result = L1.reduceByKey(g)$

- Map with f : For each triplet: outputs 2 pairs: ("price", price) and ("weight", weight).
 - e.g., $f(1, 14, 1.5) \Rightarrow [(\text{"price"}, (14,14)), (\text{"weight"}, (1.5,1.5))]$
 - $f(2, 9, 0.7) \Rightarrow [(\text{"price"}, (9,9)), (\text{"weight"}, (0.7,0.7))]$
- Shuffle: group by key:
 - $(\text{"price"}, (14,14)), (\text{"price"}, (9,9)), (\text{"price"}, (10,10))$
 - $(\text{"weight"}, (1.5,1.5)), (\text{"weight"}, (0.7,0.7)), (\text{"weight"}, (0.2,0.2))$
- Reduce with g : compute min and max for each key
 - $(\text{"price"}, (14,14)), (\text{"price"}, (9,9)), (\text{"price"}, (10,10)) \Rightarrow^g (\text{"price"}, (9, 14))$
 - $(\text{"weight"}, (1.5,1.5)), (\text{"weight"}, (0.7,0.7)), (\text{"weight"}, (0.2,0.2)) \Rightarrow^g (\text{"weight"}, (0.2, 1.5))$

Example 2: exercises

- Is it map or flatMap ? (justify)
- Write the f and g functions to compute min and max for price and weight.
- If the database has n rows, how many calls to f and g functions are done ?
- Now we want to compute the average values of price and weight. How to do it ?